Le Resume Studio

Web and Mobile Browser Guide

October 12 2026 | Published app version 14

Use this guide to investigate browser and device differences, reproduce failures, and verify the app before making compatibility claims. The app includes capability fallbacks and responsive layouts. Real Safari, Chrome, mobile, screen-reader, and live ATS behavior has not been certified by the automated checks.

1 Support model and current limits

Intended targets are current Safari on macOS/iPhone/iPad, Chrome on desktop/Android, and desktop Edge and Firefox. The app requires JavaScript modules and native HTML dialogs. It uses capability detection for optional APIs rather than promising identical controls on every browser. It is a website, with no native iOS/Android package or service worker.

Capability	Implemented behavior	Status
Folder picker	Save file… when showSaveFilePicker is available outside an embedded frame; otherwise Download file.	Simulated
File sharing	Share file only when File, share, and canShare accept the exact file, outside frames.	Simulated
Clipboard	Direct writeText on click; manual selectable-text dialog after rejection or absence.	Simulated
Browser storage	Persistent notice and current-tab operation after blocked reads/writes or full storage.	Simulated
Mobile keyboard	VisualViewport height/offset update; CSS viewport fallback; overrides removed during pinch zoom.	Simulated
Clone and syntax	JSON clone fallback; application avoids literal regex lookbehind, Array.at, and Object.hasOwn.	Simulated
Layout and accessibility	Responsive CSS, safe areas, zoom, focus, tab semantics, and reduced motion.	Code checks
Actual device interactions	Download managers, share sheets, rendering, rotation, keyboards, and screen readers.	Needs testing

Simulated means the feature or failure branch was exercised with test adapters. It does not certify a particular browser version. A supported optional API can still be denied by permissions, security policy, an embedded context, or user cancellation. Record the browser and operating system when verifying a release.

© 2026 Marabou Lane LLC

2 Download and export troubleshooting

A prepared file does not appear in Downloads

Reproduce by choosing a document and format, then inspect the save dialog. Format buttons prepare the Blob; the user must choose Save file… or Download file. In an embedded app, use Open download page. Ordinary downloads follow browser settings and may open in a preview. The app cannot set a universal destination folder. MDN documents the picker as limited in availability, requiring a secure context and user activation [1].

Inspect downloads.js configureSaveLink and canChooseLocation, plus download-page.js. Native picker success is reported only after write and close. An AbortError means cancellation; another save error switches the retained link to a normal download. Keep the Blob link alive until replacement/reset and do not automatically click then revoke it. Check browser download permissions and the preview’s own Download action.

Sharing is missing or fails

Absence of Share file is expected if exact-file sharing is unsupported or the app is embedded. configureShareButton checks canShare before displaying it and invokes share directly from a user click. A permissions policy or loss of transient activation can reject sharing [2]. Cancellation leaves the download available. Promise resolution is not treated as proof of a file saved to the device.

Master resume upload troubleshooting

Use the native file picker with DOCX, text-based PDF or UTF-8 TXT. Limits are 10 MB, 30 PDF pages and 200,000 extracted characters. Convert older .doc files; remove PDF passwords; use a text copy for scans. Review every editable field and extracted text before Save, especially for columns, tables and unusual headings. No OCR or layout-perfect conversion is promised. Errors preserve the saved master. Cancel import preserves the preceding draft; Clear master removes it and prevents pending results from restoring it.

For PDF failures, check local vendor/pdfjs/pdf.mjs, pdf.worker.mjs and standard_fonts assets, module MIME types and worker policy. For DOCX, check vendor/jszip.min.js. Readers load on demand without uploading resume contents. Modern browser file pickers may show cloud providers; the selected file must be downloadable by the browser. If a reader is unsupported or blocked, use TXT or manual entry. Native Safari, Chrome, mobile and assistive-technology upload interactions require real-device checks.

Word or PDF export fails while Plain text works

Check browser.js loadExportLibrary and network responses for vendor/docx.js or vendor/pdf-lib.min.js. Libraries are loaded on demand, concurrent requests are deduplicated, and a failed load can be retried. PDF generation also needs the vendored font programs. Serve correct JavaScript MIME types over HTTP/HTTPS and check for missing files, policy blocks, or rejected fetches. Plain text has no export-library dependency.

The PDF changes characters or the ATS rejects a file

exports.js normalizes PDF text into a limited Latin character set; characters outside it may become question marks and accents may be removed. ats.js reviews this and warns users to use Word or TXT to preserve text. An employer’s upload restrictions remain authoritative. Choose a supported file type and check the actual prepared size; the app’s profiles include source-backed limits for Greenhouse and Workable. After upload, inspect parsed fields. No live parser acceptance is guaranteed.

Acceptance check: prepare and open all six resume/cover-letter combinations, confirm filenames and content, cancel a picker, retry a failed save, and verify that each file exists in the chosen destination. Test both an ordinary browser tab and the embedded handoff.

© 2026 Marabou Lane LLC

3 Storage resets transfers and copying

Symptom	Cause or expected behavior	Developer action
Sections 1 and 2 become blank	Expected on load, reload, restart, or pageshow. Master and history stay local.	Check resetWorkspace and workspaceEpoch guards.
A new user sees someone’s master	Same browser profile or unintended published seed. A genuinely fresh profile must be empty.	Check local keys, createBlankMaster, and the deployment archive.
Saved versions are absent	Different browser/profile/origin, cleared site data, private session, or storage rejection.	Check the persistent notice and caught storage errors.
History returns after deletion	Failed persistent removal or stale cross-tab state.	Check removeItem fallback, retry status, and storage listener.
A download page says to return	Missing/invalid fragment and no valid session snapshot, or transfer exceeds the cap.	Prepare a new payload; check decode/validation and sessionStorage.
Copying is denied	Clipboard API is absent or denied. Automatic write needs an allowed secure context [3].	Use the focus-managed manual copy dialog; preserve the document text.

Browser data is isolated by context

localStorage belongs to an origin and browser profile. Private browsing has different persistence behavior, and access can throw SecurityError [4]. An embedded app may face additional storage restrictions. Treat storage as optional: the app must remain usable with its persistent warning, without claiming changes were saved when a write failed.

Transfer snapshots have a separate lifetime

The standalone download page receives only the selected document in a base64url fragment. It removes the fragment on receipt and retains a validated snapshot in that tab’s sessionStorage. It does not depend on shared localStorage with the embedded app. Clearing Saved versions resets app history and prepared links but does not erase a separate download tab’s snapshot or files already downloaded. Avoid sharing document transfer links as if they were the ordinary app link.

Acceptance check: save an application, reload and confirm blank Sections 1 and 2, explicitly reopen history, clear it, and confirm zero after reopening. Test missing master recovery from local history, explicit master clearing, a second fresh browser profile, blocked storage, clipboard denial, and deletion in another open app tab.

© 2026 Marabou Lane LLC

4 Mobile layout focus and text issues

The keyboard hides a field or save control

A phone keyboard changes the visible viewport without necessarily changing the layout viewport [5]. browser.js trackViewport listens for visible viewport resize/scroll and supplies --visible-height and --visible-top to browser.css. Dialogs use these variables with viewport fallbacks and scrolling. Check event updates while typing, rotating, and scrolling. At pinch-zoom scale other than one, overrides are removed so the browser manages zoom.

Controls overlap a notch or the screen edge

Both pages include viewport-fit=cover. browser.css adds env(safe-area-inset-*) spacing for navigation, workspace, dialogs, footers, and the download page. WebKit describes these insets for edge-to-edge layouts [6]. Test portrait and landscape, including short-height landscape navigation. Keep max()/calc() fallbacks and verify computed styles on the affected device.

Text zoom causes clipping or unwanted input zoom

Check stylesheet order and form font size. accessibility.css uses relative text sizing and controls at least 16 CSS pixels by default. Containers use min-width:0, wrap long labels and filenames, and change grid layouts on narrow screens. Do not fix clipping by disabling zoom. Verify 200% and 400% browser zoom or equivalent large-text settings, plus a 320 CSS-pixel viewport. Document previews and keyword tables may scroll; task controls must remain reachable.

Dialog focus or announcements behave incorrectly

Inspect accessibility.js and matching HTML IDs. Each dialog needs a named, focusable heading, a Close control, and status inside the modal. Errors need field association and focus; closing restores the opener. Native modal dialogs manage the interaction boundary, but test Tab/Shift+Tab containment, Escape, background interaction, and returned focus with real assistive technology. Tabs need one active item, correct controls/labels, and arrow-key behavior.

An older browser cannot start the app

The shipped implementation requires ES modules and native HTML dialogs. JavaScript-disabled users receive a notice. Existing fallbacks address optional APIs, not every old browser. Update the browser or explicitly scope and implement an accessible dialog/module fallback for a new support target. Do not claim all-browser compatibility based only on capability simulations.

Acceptance check: use a real phone keyboard in every editor, add/remove master entries, rotate the device, zoom, inspect long text and filenames, open all dialogs, and operate the review tabs with keyboard and screen reader. Test VoiceOver on Safari, TalkBack on Android, and a desktop screen reader such as NVDA with the supported browsers.

© 2026 Marabou Lane LLC

5 Device verification and issue reporting

Automated evidence for this release is 107 passing tests, including upload, save, clear and real Word/PDF extraction checks. Actual device rendering, native save/share experiences, assistive technology, and uploads into the listed ATS systems remain unverified. Record results before advertising complete compatibility or accessibility conformance.

Test environment	Priority journeys
Safari on macOS	Fresh master, complete tailoring/editing, all formats, clipboard fallback, zoom, keyboard and VoiceOver.
Safari on iPhone and iPad	Keyboard viewport, rotation, safe areas, large text, downloads/previews, Share file if offered, VoiceOver.
Chrome on Android	Same document flow, Downloads, Share file if offered, keyboard/orientation, large text, TalkBack.
Chrome Edge Firefox desktop	All document formats, picker/download branches where available, keyboard, storage failures, screen-reader checks.
Embedded and restricted contexts	Exact-document handoff, denied storage/copy/share, reset during generation/export, cancellation and retries.

Release acceptance sequence

1. Start with an empty profile; verify a blank Master resume and zero versions. Add your own test master, including multiple roles, skills, and education.

2. Tailor a supported posting and a posting with an unsupported requirement. Verify separate scores and truthful gaps, and inspect all six review tabs.

3. Edit both documents, export all formats, inspect the saved files, and test copying, handoff, sharing, cancellation, and retry.

4. Reload, reopen a saved application, clear history, clear the master, and verify browser-local isolation with another profile.

5. Repeat at narrow widths, with rotation, keyboard, large text, reduced motion, forced colors where available, and assistive technology.

A useful issue report includes release version, device, OS and browser versions, embedded/direct context, storage mode, reproduction steps, expected and actual behavior, exact error, format/file size, and a sanitized example. Add severity, the affected code path, workaround, and the verified fix. Remove personal information from logs, screenshots, links, and fixtures shared with others.

Primary references reviewed October 12 2026

[1] MDN showSaveFilePicker

[2] MDN Navigator share

[3] MDN Clipboard writeText

[4] MDN localStorage

[5] MDN VisualViewport

[6] WebKit safe areas for iPhone